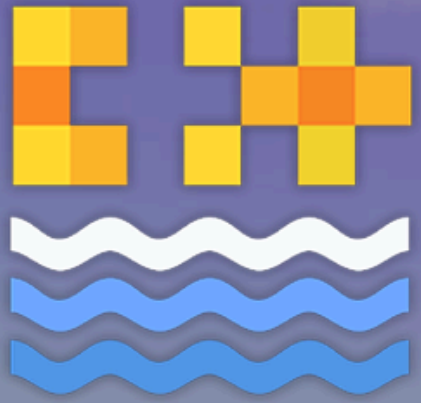




How Symbols Work and Why We Need Them

Anders Schau Knatten

2024

ASCENIUM



CPPQUIZ.ORG

C++ Quiz

You've answered 1 of 138 questions correctly. ([Clear](#))

Question #197 Difficulty: ●●●

According to the C++17 standard, what is the output of this program?

```
#include <iostream>

int j = 1;

int main() {
    int& i = j, j;
    j = 2;
    std::cout << i << j;
}
```

Answer:

Problems? View a [hint](#) or try [another question](#).

[I give up, show me the answer](#) (make 3 more attempts first).

Mode : Training

You are currently in training mode, answering random questions. Why not [Start a new quiz?](#) Then you can boast about your score, and invite your friends.

Contribute

[Create your own!](#)

Android app

Get Sergey Vasilchenko's [CppQuiz Android app](#).

The
Pragmatic
Programmers

C++ Brain Teasers

Exercise Your Mind

30 % off:
C++BrainTeasers30



Anders Schau Knatten

Foreword by Olve Maudal

Edited by Sandra Williams

HOW SYMBOLS WORK AND WHY WE NEED THEM

HOW SYMBOLS WORK AND WHY WE NEED THEM

- ELF/Linux

HOW SYMBOLS WORK AND WHY WE NEED THEM

- ELF/Linux
- C / C++

HOW SYMBOLS WORK AND WHY WE NEED THEM

HOW SYMBOLS WORK AND WHY WE NEED THEM

- Static vs. dynamic symbols

HOW SYMBOLS WORK AND WHY WE NEED THEM

- Static vs. dynamic symbols
- Binding vs. visibility (and linkage)

HOW SYMBOLS WORK AND WHY WE NEED THEM

- Static vs. dynamic symbols
- Binding vs. visibility (and linkage)
- Linking

HOW SYMBOLS WORK AND WHY WE NEED THEM

- Static vs. dynamic symbols
- Binding vs. visibility (and linkage)
- Linking
- Weak symbols

HOW SYMBOLS WORK AND WHY WE NEED THEM

- Static vs. dynamic symbols
- Binding vs. visibility (and linkage)
- Linking
- Weak symbols
- Templates and symbols

HOW SYMBOLS WORK AND WHY WE NEED THEM

- Static vs. dynamic symbols
- Binding vs. visibility (and linkage)
- Linking
- Weak symbols
- Templates and symbols
- Inline variables

HOW SYMBOLS WORK AND WHY WE NEED THEM

- Static vs. dynamic symbols
- Binding vs. visibility (and linkage)
- Linking
- Weak symbols
- Templates and symbols
- Inline variables
- Where symbols exist

HOW SYMBOLS WORK AND WHY WE NEED THEM

- Static vs. dynamic symbols
- Binding vs. visibility (and linkage)
- Linking
- Weak symbols
- Templates and symbols
- Inline variables
- Where symbols exist
- How to view symbols

HOW SYMBOLS WORK AND WHY WE NEED THEM

- Static vs. dynamic symbols
- Binding vs. visibility (and linkage)
- Linking
- Weak symbols
- Templates and symbols
- Inline variables
- Where symbols exist
- How to view symbols
- Why there are three symbol tables

HOW SYMBOLS WORK AND WHY WE NEED THEM

- Static vs. dynamic symbols
- Binding vs. visibility (and linkage)
- Linking
- Weak symbols
- Templates and symbols
- Inline variables
- Where symbols exist
- How to view symbols
- Why there are three symbol tables
- Stripping symbols

HOW SYMBOLS WORK AND WHY WE NEED THEM

- Static vs. dynamic symbols
- Binding vs. visibility (and linkage)
- Linking
- Weak symbols
- Templates and symbols
- Inline variables
- Where symbols exist
- How to view symbols
- Why there are three symbol tables
- Stripping symbols
- Who gets a symbol

HOW SYMBOLS WORK AND WHY WE NEED THEM

HOW SYMBOLS WORK AND WHY WE NEED THEM

- Investigating linker errors

HOW SYMBOLS WORK AND WHY WE NEED THEM

- Investigating linker errors
- Consuming libraries

HOW SYMBOLS WORK AND WHY WE NEED THEM

- Investigating linker errors
- Consuming libraries
- Shipping binaries, especially libraries

HOW SYMBOLS WORK AND WHY WE NEED THEM

- Investigating linker errors
- Consuming libraries
- Shipping binaries, especially libraries
- Understanding your binaries

A MOTIVATING EXAMPLE

What are symbols ?

A MOTIVATING EXAMPLE

What are symbols for?

values.c

```
int value = 42;  
  
int getValue() {  
    return value;  
}
```


values.c

```
int value = 42;  
  
int getValue() {  
    return value;  
}
```

main.c

```
#include "values.h"  
  
int main() {  
    return getValue();  
}
```


values.c

```
int value = 42;  
  
int getValue() {  
    return value;  
}
```

main.c

```
#include "values.h"  
  
int main() {  
    return getValue();  
}
```

values.o

2a	00	00	00
55	48	89	e5
8b	05	00	00
00	00	5d	c3

main.o

0f	d6	23	04
11	f8	d1	c3
87	03	12	7a
3b	48	28	23
fe	e9	63	c6

values.c

```
int value = 42;  
  
int getValue() {  
    return value;  
}
```

main.c

```
#include "values.h"  
  
int main() {  
    return getValue();  
}
```

values.o

2a	00	00	00
55	48	89	e5
8b	05	00	00
00	00	5d	c3

main.o

0f	d6	23	04
11	f8	d1	c3
87	03	12	7a
3b	48	28	23
fe	e9	63	c6

library.a

83	75	03	4f
8f	20	a7	37
83	74	9c	39
27	37	ff	a8
34	9c	61	23
87	fa	ca	72
94	17	53	93
9b	be	38	48

values.c

```
int value = 42;  
  
int getValue() {  
    return value;  
}
```

main.c

```
#include "values.h"  
  
int main() {  
    return getValue();  
}
```

values.o

2a	00	00	00
55	48	89	e5
8b	05	00	00
00	00	5d	c3

main.o

0f	d6	23	04
11	f8	d1	c3
87	03	12	7a
3b	48	28	23
fe	e9	63	c6

library.a

83	75	03	4f
8f	20	a7	37
83	74	9c	39
27	37	ff	a8
34	9c	61	23
87	fa	ca	72
94	17	53	93
9b	be	38	48

the_program

values.c

```
int value = 42;  
  
int getValue() {  
    return value;  
}
```

main.c

```
#include "values.h"  
  
int main() {  
    return getValue();  
}
```

values.o

2a	00	00	00
55	48	89	e5
8b	05	00	00
00	00	5d	c3

main.o

0f	d6	23	04
11	f8	d1	c3
87	03	12	7a
3b	48	28	23
fe	e9	63	c6

library.a

83	75	03	4f
8f	20	a7	37
83	74	9c	39
27	37	ff	a8
34	9c	61	23
87	fa	ca	72
94	17	53	93
9b	be	38	48

the_program

2a	00	00	00
0f	d6	23	04
83	75	03	4f
8f	20	a7	37

values.c

```
int value = 42;  
  
int getValue() {  
    return value;  
}
```

main.c

```
#include "values.h"  
  
int main() {  
    return getValue();  
}
```

values.o

2a	00	00	00
55	48	89	e5
8b	05	00	00
00	00	5d	c3

main.o

0f	d6	23	04
11	f8	d1	c3
87	03	12	7a
3b	48	28	23
fe	e9	63	c6

library.a

83	75	03	4f
8f	20	a7	37
83	74	9c	39
27	37	ff	a8
34	9c	61	23
87	fa	ca	72
94	17	53	93
9b	be	38	48

the_program

2a	00	00	00
0f	d6	23	04
83	75	03	4f
8f	20	a7	37
55	48	89	e5
8b	05	00	00
00	00	5d	c3
11	f8	d1	c3
87	03	12	7a
3b	48	28	23
fe	e9	63	c6
83	74	9c	39
27	37	ff	a8
34	9c	61	23
87	fa	ca	72
94	17	53	93
9b	be	38	48

values.c

```
int value = 42;  
  
int getValue() {  
    return value;  
}
```

values.o

2a	00	00	00
55	48	89	e5
8b	05	00	00
00	00	5d	c3

values.c

```
int value = 42;  
  
int getValue() {  
    return value;  
}
```

```
1 $ objdump -s values.o  
2 values.o: file format elf64-x86-64  
3  
4 Contents of section .text:  
5 0000 554889e5 8b050000 00005dc3  
6 Contents of section .data:  
7 0000 2a000000  
8 (...)
```

values.c

```
int value = 42;  
  
int getValue() {  
    return value;  
}
```

```
1 $ objdump -s values.o  
2 values.o: file format elf64-x86-64  
3  
4 Contents of section .text:  
5 0000 554889e5 8b050000 00005dc3  
6 Contents of section .data:  
7 0000 2a000000  
8 (...)
```



```
1 $ objdump -d values.o
2
3 values.o:      file format elf64-x86-64
4
5 Disassembly of section .text:
6
7 0000000000000000 <getValue>:
8   0:      55                push    %rbp
9   1:      48 89 e5          mov     %rsp,%rbp
10  4:      8b 05 00 00 00 00  mov     0x0(%rip),%eax      # a <getValue+0xa>
11  a:      5d                pop     %rbp
12  b:      c3                retq
13
14 ( ... )
```

```
1 $ objdump -d values.o
2
3 values.o:      file format elf64-x86-64
4
5 Disassembly of section .text:
6
7 0000000000000000 <getValue>:
8   0:      55                push    %rbp
9   1:      48 89 e5          mov     %rsp,%rbp
10  4:      8b 05 00 00 00 00  mov     0x0(%rip),%eax      # a <getValue+0xa>
11  a:      5d                pop     %rbp
12  b:      c3            retq
13
14 ( ... )
```

```
1 $ objdump -t values.o
2 values.o:      file format elf64-x86-64
3
4 SYMBOL TABLE:
5 (...)
6 0000000000000000 g      O .data      0000000000000004 value
7 0000000000000000 g      F .text      000000000000000c getValue
```



```
1 $ objdump -t values.o
2 values.o:      file format elf64-x86-64
3
4 SYMBOL TABLE:
5 (...)
6 0000000000000000 g      O .data      0000000000000004 value
7 0000000000000000 g      F .text      000000000000000c getValue
```

```
1 $ objdump -t values.o
2 values.o:      file format elf64-x86-64
3
4 SYMBOL TABLE:
5 (...)
6 0000000000000000 g      O .data      0000000000000004 value
7 0000000000000000 g      F .text      000000000000000c getValue
```

```
$ nm values.o
0000000000000000 T getValue
0000000000000000 D value
```

```
1 $ objdump -t values.o
2 values.o:      file format elf64-x86-64
3
4 SYMBOL TABLE:
5 (...)
6 0000000000000000 g      O .data      0000000000000004 value
7 0000000000000000 g      F .text      000000000000000c getValue
```

```
$ nm values.o
0000000000000000 T getValue
0000000000000000 D value
```

```
1 $ objdump -t main.o
2
3 main.o:      file format elf64-x86-64
4
5 SYMBOL TABLE:
6 (...)
7 0000000000000000 g      F .text      0000000000000010 main
8 0000000000000000      *UND*      0000000000000000 getValue
```

```
1 $ objdump -t values.o
2 values.o:      file format elf64-x86-64
3
4 SYMBOL TABLE:
5 (...)
6 0000000000000000 g      O .data      0000000000000004 value
7 0000000000000000 g      F .text      000000000000000c getValue
```

```
$ nm values.o
0000000000000000 T getValue
0000000000000000 D value
```

```
1 $ objdump -t main.o
2
3 main.o:      file format elf64-x86-64
4
5 SYMBOL TABLE:
6 (...)
7 0000000000000000 g      F .text      0000000000000010 main
8 0000000000000000      *UND*      0000000000000000 getValue
```

```
1 $ objdump -t values.o
2 values.o:      file format elf64-x86-64
3
4 SYMBOL TABLE:
5 (...)
6 0000000000000000 g      O .data      0000000000000004 value
7 0000000000000000 g      F .text      000000000000000c getValue
```

```
$ nm values.o
0000000000000000 T getValue
0000000000000000 D value
```

```
1 $ objdump -t main.o
2
3 main.o:      file format elf64-x86-64
4
5 SYMBOL TABLE:
6 (...)
7 0000000000000000 g      F .text      0000000000000010 main
8 0000000000000000      *UND*      0000000000000000 getValue
```

```
1 $ objdump -t values.o
2 values.o:      file format elf64-x86-64
3
4 SYMBOL TABLE:
5 (...)
6 0000000000000000 g      O .data      0000000000000004 value
7 0000000000000000 g      F .text      000000000000000c getValue
```

```
$ nm values.o
0000000000000000 T getValue
0000000000000000 D value
```

```
1 $ objdump -t main.o
2
3 main.o:      file format elf64-x86-64
4
5 SYMBOL TABLE:
6 (...)
7 0000000000000000 g      F .text      0000000000000010 main
8 0000000000000000      *UND*      0000000000000000 getValue
```



```
1 $ objdump -t values.o
2 values.o:      file format elf64-x86-64
3
4 SYMBOL TABLE:
5 (...)
6 0000000000000000 g      O .data      0000000000000004 value
7 0000000000000000 g      F .text      000000000000000c getValue
```

```
$ nm values.o
0000000000000000 T getValue
0000000000000000 D value
```

```
1 $ objdump -t main.o
2
3 main.o:      file format elf64-x86-64
4
5 SYMBOL TABLE:
6 (...)
7 0000000000000000 g      F .text      0000000000000010 main
8 0000000000000000      *UND*      0000000000000000 getValue
```

```
$ nm main.o
(...)
                U getValue
0000000000000000 T main
```



```
1 $ readelf -SW values.o
2 There are 20 section headers, starting at offset 0x610:
3
4 Section Headers:
5   [Nr] Name                Type          Address          Off    Size    ES Flg I
6   [ 0]                     NULL          0000000000000000 000000 000000 00
7   [ 1] .text                   PROGBITS      0000000000000000 000040 00000c 00  AX
8   [ 2] .rela.text             RELA          0000000000000000 000400 000018 18  I 1
9   [ 3] .data                   PROGBITS      0000000000000000 00004c 000004 00  WA
10  [ 4] .bss                    NOBITS        0000000000000000 000050 000000 00  WA
11  [ 5] .debug_info              PROGBITS      0000000000000000 000050 000067 00
12  [ 6] .rela.debug_info        RELA          0000000000000000 000418 0000f0 18  I 1
13  [ 7] .debug_abbrev            PROGBITS      0000000000000000 0000b7 000048 00
14  [ 8] .debug_aranges           PROGBITS      0000000000000000 0000ff 000030 00
15  [ 9] .rela.debug_aranges     RELA          0000000000000000 000508 000030 18  I 1
```

11	[5]	.debug_info	PROGBITS	0000000000000000	000050	000067	00		
12	[6]	.rela.debug_info	RELA	0000000000000000	000418	0000f0	18	I	1
13	[7]	.debug_abbrev	PROGBITS	0000000000000000	0000b7	000048	00		
14	[8]	.debug_aranges	PROGBITS	0000000000000000	0000ff	000030	00		
15	[9]	.rela.debug_aranges	RELA	0000000000000000	000508	000030	18		1
16	[10]	.debug_line	PROGBITS	0000000000000000	00012f	00005c	00		
17	[11]	.rela.debug_line	RELA	0000000000000000	000538	000018	18	I	1
18	[12]	.debug_str	PROGBITS	0000000000000000	00018b	000091	01	MS	
19	[13]	.comment	PROGBITS	0000000000000000	00021c	00002a	01	MS	
20	[14]	.note.GNU-stack	PROGBITS	0000000000000000	000246	000000	00		
21	[15]	.eh_frame	PROGBITS	0000000000000000	000248	000038	00	A	
22	[16]	.rela.eh_frame	RELA	0000000000000000	000550	000018	18	I	1
23	[17]	.symtab	SYMTAB	0000000000000000	000280	000168	18		1
24	[18]	.strtab	STRTAB	0000000000000000	0003e8	000016	00		
25	[19]	.shstrtab	STRTAB	0000000000000000	000568	0000a8	00		

main.c

```
#include "values.h"

int main() {
    return getValue();
}
```

main.o

0f	d6	23	04
11	f8	d1	c3
87	03	12	7a
3b	48	28	23
fe	e9	63	c6

values.c

```
int value = 42;

int getValue() {
    return value;
}
```

main.c

```
#include "values.h"

int main() {
    return getValue();
}
```

values.o

2a	00	00	00
55	48	89	e5
8b	05	00	00
00	00	5d	c3

main.o

0f	d6	23	04
11	f8	d1	c3
87	03	12	7a
3b	48	28	23
fe	e9	63	c6

library.a

83	75	03	4f
8f	20	a7	37
83	74	9c	39
27	37	ff	a8
34	9c	61	23
87	fa	ca	72
94	17	53	93
9b	be	38	48

the_program

2a	00	00	00
0f	d6	23	04
83	75	03	4f
8f	20	a7	37
55	48	89	e5
8b	05	00	00
00	00	5d	c3
11	f8	d1	c3
87	03	12	7a
3b	48	28	23
fe	e9	63	c6
83	74	9c	39
27	37	ff	a8
34	9c	61	23
87	fa	ca	72
94	17	53	93
9b	be	38	48

RELOCATIONS

```
objdump -dr -M intel main.o
```


RELOCATIONS

```
objdump -dr -M intel main.o
```

```
1 Disassembly of section .text:
2
3 0000000000000000 <main>:
4   0:   55                push    rbp
5   1:   48 89 e5          mov     rbp, rsp
6   4:   b8 00 00 00 00    mov     eax, 0x0
7   9:   e8 00 00 00 00    call    e <main+0xe>
8               a: R_X86_64_PLT32  getValue-0x4
9   e:   5d                pop     rbp
10  f:   c3                ret
```


RELOCATIONS

```
objdump -dr -M intel main.o
```

```
1 Disassembly of section .text:
2
3 0000000000000000 <main>:
4   0:   55                push    rbp
5   1:   48 89 e5          mov     rbp, rsp
6   4:   b8 00 00 00 00    mov     eax, 0x0
7   9:   e8 00 00 00 00    call    e <main+0xe>
8               a: R_X86_64_PLT32  getValue-0x4
9   e:   5d                pop     rbp
10  f:   c3                ret
```

RELOCATIONS

```
objdump -dr -M intel main.o
```

```
1 Disassembly of section .text:
2
3 0000000000000000 <main>:
4   0:   55                push    rbp
5   1:   48 89 e5          mov     rbp, rsp
6   4:   b8 00 00 00 00    mov     eax, 0x0
7   9:   e8 00 00 00 00    call    e <main+0xe>
8             a: R_X86_64_PLT32    getValue-0x4
9   e:   5d                pop     rbp
10  f:   c3                ret
```

- Linker loops over `main.o` relocation table

RELOCATIONS

```
objdump -dr -M intel main.o
```

```
1 Disassembly of section .text:
2
3 0000000000000000 <main>:
4   0:   55                push    rbp
5   1:   48 89 e5          mov     rbp, rsp
6   4:   b8 00 00 00 00    mov     eax, 0x0
7   9:   e8 00 00 00 00    call    e <main+0xe>
8             a: R_X86_64_PLT32    getValue-0x4
9   e:   5d                pop     rbp
10  f:   c3                ret
```

- Linker loops over `main.o` relocation table
- Looks in `values.o` symbol table to find `getValue`

RELOCATIONS

```
objdump -dr -M intel main.o
```

```
1 Disassembly of section .text:
2
3 0000000000000000 <main>:
4   0:   55                push    rbp
5   1:   48 89 e5          mov     rbp, rsp
6   4:   b8 00 00 00 00    mov     eax, 0x0
7   9:   e8 00 00 00 00    call    e <main+0xe>
8             a: R_X86_64_PLT32    getValue-0x4
9   e:   5d                pop     rbp
10  f:   c3                ret
```

- Linker loops over `main.o` relocation table
- Looks in `values.o` symbol table to find `getValue`
- Knows where in `the_program` it put `.text` from `values.o`

RELOCATIONS

```
objdump -dr -M intel main.o
```

```
1 Disassembly of section .text:
2
3 0000000000000000 <main>:
4   0:   55                push    rbp
5   1:   48 89 e5          mov     rbp, rsp
6   4:   b8 00 00 00 00    mov     eax, 0x0
7   9:   e8 00 00 00 00    call    e <main+0xe>
8             a: R_X86_64_PLT32    getValue-0x4
9   e:   5d                pop     rbp
10  f:   c3                ret
```

- Linker loops over `main.o` relocation table
- Looks in `values.o` symbol table to find `getValue`
- Knows where in the program it put `.text` from `values.o`
- Computes the final address of `getValue`

RELOCATIONS

```
objdump -dr -M intel main.o
```

```
1 Disassembly of section .text:
2
3 0000000000000000 <main>:
4   0:   55                push    rbp
5   1:   48 89 e5          mov     rbp, rsp
6   4:   b8 00 00 00 00    mov     eax, 0x0
7   9:   e8 00 00 00 00    call    e <main+0xe>
8             a: R_X86_64_PLT32    getValue-0x4
9   e:   5d                pop     rbp
10  f:   c3                ret
```

- Linker loops over `main.o` relocation table
- Looks in `values.o` symbol table to find `getValue`
- Knows where in the program it put `.text` from `values.o`
- Computes the final address of `getValue`
- Fills it in here

FINAL BINARY

```
objdump -d -M intel the_program
```


FINAL BINARY

```
objdump -d -M intel the_program
```

```
1 00000000000000606 <main>:
2 606: 55                push    rbp
3 607: 48 89 e5          mov     rbp, rsp
4 60a: b8 00 00 00 00    mov     eax, 0x0
5 60f: e8 e6 ff ff ff    call    5fa <getValue>
6 614: 5d                pop     rbp
7 615: c3                ret
```


FINAL BINARY

```
objdump -d -M intel the_program
```

```
1 00000000000000606 <main>:
2 606: 55                push    rbp
3 607: 48 89 e5          mov     rbp, rsp
4 60a: b8 00 00 00 00    mov     eax, 0x0
5 60f: e8 e6 ff ff ff    call    5fa <getValue>
6 614: 5d                pop     rbp
7 615: c3                ret
```

FINAL BINARY

```
objdump -d -M intel the_program
```

```
1 00000000000000606 <main>:
2 606: 55                push    rbp
3 607: 48 89 e5          mov     rbp, rsp
4 60a: b8 00 00 00 00    mov     eax, 0x0
5 60f: e8 e6 ff ff ff    call    5fa <getValue>
6 614: 5d                pop     rbp
7 615: c3                ret
```

IF THERE WERE NO SYMBOLS

IF THERE WERE NO SYMBOLS

```
1 $ gcc -o the_program values.o main.o
2 $ ./the_program ; echo $?
3 42
4 $ nm values.o
5 0000000000000000 T getValue
6 0000000000000000 D value
7 $ strip values.o
8 $ nm values.o
9 nm: values.o: no symbols
10 $ gcc -o the_program values.o main.o
11 /usr/bin/ld: error in values.o(.eh_frame); no .eh_frame_hdr table will be created
12 main.o: In function `main':
13 main.c:(.text+0xa): undefined reference to `getValue'
14 collect2: error: ld returned 1 exit status
```

IF THERE WERE NO SYMBOLS

```
1 $ gcc -o the_program values.o main.o
2 $ ./the_program ; echo $?
3 42
4 $ nm values.o
5 0000000000000000 T getValue
6 0000000000000000 D value
7 $ strip values.o
8 $ nm values.o
9 nm: values.o: no symbols
10 $ gcc -o the_program values.o main.o
11 /usr/bin/ld: error in values.o(.eh_frame); no .eh_frame_hdr table will be created
12 main.o: In function `main':
13 main.c:(.text+0xa): undefined reference to `getValue'
14 collect2: error: ld returned 1 exit status
```

IF THERE WERE NO SYMBOLS

```
1 $ gcc -o the_program values.o main.o
2 $ ./the_program ; echo $?
3 42
4 $ nm values.o
5 0000000000000000 T getValue
6 0000000000000000 D value
7 $ strip values.o
8 $ nm values.o
9 nm: values.o: no symbols
10 $ gcc -o the_program values.o main.o
11 /usr/bin/ld: error in values.o(.eh_frame); no .eh_frame_hdr table will be created
12 main.o: In function `main':
13 main.c:(.text+0xa): undefined reference to `getValue'
14 collect2: error: ld returned 1 exit status
```

IF THERE WERE NO SYMBOLS

```
1 $ gcc -o the_program values.o main.o
2 $ ./the_program ; echo $?
3 42
4 $ nm values.o
5 0000000000000000 T getValue
6 0000000000000000 D value
7 $ strip values.o
8 $ nm values.o
9 nm: values.o: no symbols
10 $ gcc -o the_program values.o main.o
11 /usr/bin/ld: error in values.o(.eh_frame); no .eh_frame_hdr table will be created
12 main.o: In function `main':
13 main.c:(.text+0xa): undefined reference to `getValue'
14 collect2: error: ld returned 1 exit status
```


IF THERE WERE NO SYMBOLS

```
1 $ gcc -o the_program values.o main.o
2 $ ./the_program ; echo $?
3 42
4 $ nm values.o
5 0000000000000000 T getValue
6 0000000000000000 D value
7 $ strip values.o
8 $ nm values.o
9 nm: values.o: no symbols
10 $ gcc -o the_program values.o main.o
11 /usr/bin/ld: error in values.o(.eh_frame); no .eh_frame_hdr table will be created
12 main.o: In function `main':
13 main.c:(.text+0xa): undefined reference to `getValue'
14 collect2: error: ld returned 1 exit status
```


IF THERE WERE NO SYMBOLS

```
1 $ gcc -o the_program values.o main.o
2 $ ./the_program ; echo $?
3 42
4 $ nm values.o
5 0000000000000000 T getValue
6 0000000000000000 D value
7 $ strip values.o
8 $ nm values.o
9 nm: values.o: no symbols
10 $ gcc -o the_program values.o main.o
11 /usr/bin/ld: error in values.o(.eh_frame); no .eh_frame_hdr table will be created
12 main.o: In function `main':
13 main.c:(.text+0xa): undefined reference to `getValue'
14 collect2: error: ld returned 1 exit status
```

THE STORY SO FAR

THE STORY SO FAR

- Separate compilation model

THE STORY SO FAR

- Separate compilation model
- Sections `.data`, `.text`, `.symtab` etc

THE STORY SO FAR

- Separate compilation model
- Sections `.data`, `.text`, `.symtab` etc
- `.text` only has machine code

THE STORY SO FAR

- Separate compilation model
- Sections `.data`, `.text`, `.symtab` etc
- `.text` only has machine code
- `.data` only has data

THE STORY SO FAR

- Separate compilation model
- Sections `.data`, `.text`, `.symtab` etc
- `.text` only has machine code
- `.data` only has data
- `.symtab` has symbols pointing to each function/object

THE STORY SO FAR

- Separate compilation model
- Sections `.data`, `.text`, `.symtab` etc
- `.text` only has machine code
- `.data` only has data
- `.symtab` has symbols pointing to each function/object
- Compiler creates relocations in separate section

THE STORY SO FAR

- Separate compilation model
- Sections `.data`, `.text`, `.symtab` etc
- `.text` only has machine code
- `.data` only has data
- `.symtab` has symbols pointing to each function/object
- Compiler creates relocations in separate section
- Linker fills in relocations

THE STORY SO FAR

- Separate compilation model
- Sections `.data`, `.text`, `.symtab` etc
- `.text` only has machine code
- `.data` only has data
- `.symtab` has symbols pointing to each function/object
- Compiler creates relocations in separate section
- Linker fills in relocations
- Can't link without symbols

LINKAGE

LINKAGE

```
$ cat values.c
int value = 42;

static int helper() {
    return value;
}

int getValue() {
    return helper();
}
```

LINKAGE

```
$ cat values.c
int value = 42;

static int helper() {
    return value;
}

int getValue() {
    return helper();
}
```

```
$ nm values.o
0000000000000000c T getValue
00000000000000000 t helper
00000000000000000 D value
```

LINKAGE

```
$ cat values.c
int value = 42;

static int helper() {
    return value;
}

int getValue() {
    return helper();
}
```

```
$ nm values.o
0000000000000000c T getValue
00000000000000000 t helper
00000000000000000 D value
```

C/C++ Linkage

value	external
helper	internal
getValue	external

LINKAGE

```
$ cat values.c
int value = 42;

static int helper() {
    return value;
}

int getValue() {
    return helper();
}
```

```
$ nm values.o
0000000000000000c T getValue
00000000000000000 t helper
00000000000000000 D value
```

	C/C++ Linkage	ELF Binding
value	external	global
helper	internal	local
getValue	external	global

LINKAGE

```
1 $ cat values.c
2 int value = 42;
3
4 static int helper() {
5     return value;
6 }
7
8 int getValue() {
9     return helper();
10 }
```

```
$ nm values.o
0000000000000000c T getValue
00000000000000000 t helper
00000000000000000 D value
```

LINKAGE

```
1 $ cat values.c
2 int value = 42;
3
4 static int helper() {
5     return value;
6 }
7
8 int getValue() {
9     return helper();
10 }
```

```
$ nm values.o
0000000000000000c T getValue
0000000000000000 t helper
0000000000000000 D value
```

```
1 $ cat main.c
2 #include "values.h"
3 #include <stdio.h>
4
5 static void helper() {
6     printf("USAGE: ...");
7 }
8
9 int main() {
10     return getValue();
11 }
```

```
$ nm main.o
0000000000000000c T getValue
0000000000000000 t helper
0000000000000000 D value
```

LINKAGE

```
1 $ cat values.c
2 int value = 42;
3
4 static int helper() {
5     return value;
6 }
7
8 int getValue() {
9     return helper();
10 }
```

```
$ nm values.o
0000000000000000c T getValue
0000000000000000 t helper
0000000000000000 D value
```

```
1 $ cat main.c
2 #include "values.h"
3 #include <stdio.h>
4
5 static void helper() {
6     printf("USAGE: ...");
7 }
8
9 int main() {
10     return getValue();
11 }
```

```
$ nm main.o
0000000000000000c T getValue
0000000000000000 t helper
0000000000000000 D value
```

HEADER FILES

```
$ cat values.h
#pragma once

int value = 42;

int getValue() {
    return value;
}
```

```
$ cat values.c
#include "values.h"
```

```
$ cat main.c
#include "values.h"

int main() {
    return getValue();
}
```

HEADER FILES

```
$ cat values.h
#pragma once

int value = 42;

int getValue() {
    return value;
}
```

```
$ cat values.c
#include "values.h"
```

```
$ cat main.c
#include "values.h"

int main() {
    return getValue();
}
```

```
values.h:6: multiple definition of `value'
values.h:8: multiple definition of `getValue'
```

HEADER FILES

```
$ cat values.h
#pragma once

extern int value;

int getValue();
```

```
$ cat values.c
#include "values.h"

int value = 42;

int getValue() {
    return value;
}
```

```
$ cat main.c
#include "values.h"

int main() {
    return getValue();
}
```

C++ HEADERS

C++ HEADERS

```
$ cat values.h
#pragma once

struct SomeClass
{
    int getValue() { return 2; }
};
```


C++ HEADERS

```
$ cat values.h
#pragma once

struct SomeClass
{
    int getValue() { return 2; }
};

template<typename T>
T getValue() {
    return 1;
}
```

C++ HEADERS

```
$ cat values.h
```

```
#pragma once
```

```
struct SomeClass {  
    int getValue() { return 2; }  
};
```

```
template<typename T>  
T getValue() {  
    return 1;  
}
```

```
$ cat values.cpp
```

```
#include "values.h"
```

```
int useIt() {  
    return getValue<int>() + SomeClass{}.getValue();  
}
```

C++ HEADERS

```
$ cat values.h
#pragma once

struct SomeClass {
    int getValue() { return 2; }
};

template<typename T>
T getValue() {
    return 1;
}
```

```
$ cat values.cpp
#include "values.h"

int useIt() {
    return getValue<int>() + SomeClass{}.getValue();
}
```

```
$ nm -C values.o
0000000000000000 W int getValue<int>()
0000000000000000 T useIt()
0000000000000000 W SomeClass::getValue()
```

WEAK SYMBOLS

WEAK SYMBOLS

```
$ cat values.h  
#pragma once  
  
struct Resource *loadResource();
```

WEAK SYMBOLS

```
$ cat values.h
#pragma once

struct Resource *loadResource();
```

```
$ cat values.c
#include "values.h"

struct Resource* loadResource() {/**/}
```

WEAK SYMBOLS

```
$ cat values.h
#pragma once

struct Resource *loadResource();
```

```
$ cat values.c
#include "values.h"

struct Resource* loadResource() {/**/}
```

```
1 $ cat main.c
2 #include "values.h"
3
4 int main() {
5     struct Resource* resource = loadResource();
6     return resource->value;
7 }
```


WEAK SYMBOLS

```
$ cat values.h
#pragma once

struct Resource *loadResource();
```

```
$ cat values.c
#include "values.h"

struct Resource* loadResource() {/**/}
```

```
1 $ cat main.c
2 #include "values.h"
3
4 int main() {
5     struct Resource* resource = loadResource();
6     return resource->value;
7 }
```


WEAK SYMBOLS

```
$ cat values.h
#pragma once

struct Resource *loadResource();
```

```
$ cat values.c
#include "values.h"

struct Resource* loadResource() {/**/}
```

```
1 $ cat main.c
2 #include "values.h"
3
4 int main() {
5     struct Resource* resource = loadResource();
6     return resource->value;
7 }
```

WEAK SYMBOLS

```
$ cat values.h
```

```
#pragma once
```

```
struct Resource *loadResource();
```

```
1 $ cat values.c
```

```
2 #include "values.h"
```

```
3
```

```
4 __attribute__((weak)) struct Resource* loadResource() {/**/}
```

WEAK SYMBOLS

```
$ cat values.h
```

```
#pragma once
```

```
struct Resource *loadResource();
```

```
1 $ cat values.c
```

```
2 #include "values.h"
```

```
3
```

```
4 __attribute__((weak)) struct Resource* loadResource() {/**/}
```

WEAK SYMBOLS

```
$ cat values.h
```

```
#pragma once
```

```
struct Resource *loadResource();
```

```
1 $ cat values.c
```

```
2 #include "values.h"
```

```
3
```

```
4 __attribute__((weak)) struct Resource* loadResource() {/**/}
```

```
1 $ cat main.c
```

```
2 #include "values.h"
```

```
3
```

```
4 struct Resource* loadResource() {/**/}
```

```
5
```

```
6 int main() {
```

```
7     struct Resource* resource = loadResource();
```

```
8     return resource->value;
```

```
9 }
```

WEAK SYMBOLS

```
$ cat values.h
```

```
#pragma once
```

```
struct Resource *loadResource();
```

```
1 $ cat values.c
```

```
2 #include "values.h"
```

```
3
```

```
4 __attribute__((weak)) struct Resource* loadResource() {/**/}
```

```
1 $ cat main.c
```

```
2 #include "values.h"
```

```
3
```

```
4 struct Resource* loadResource() {/**/}
```

```
5
```

```
6 int main() {
```

```
7     struct Resource* resource = loadResource();
```

```
8     return resource->value;
```

```
9 }
```

WEAK SYMBOLS

```
$ cat values.h
```

```
#pragma once
```

```
struct Resource *loadResource();
```

```
1 $ cat values.c
```

```
2 #include "values.h"
```

```
3
```

```
4 __attribute__((weak)) struct Resource* loadResource() {/**/}
```

```
1 $ cat main.c
```

```
2 #include "values.h"
```

```
3
```

```
4 struct Resource* loadResource() {/**/}
```

```
5
```

```
6 int main() {
```

```
7     struct Resource* resource = loadResource();
```

```
8     return resource->value;
```

```
9 }
```

WEAK SYMBOLS

```
$ cat values.h
```

```
#pragma once
```

```
struct Resource *loadResource();
```

```
1 $ cat values.c
```

```
2 #include "values.h"
```

```
3
```

```
4 __attribute__((weak)) struct Resource* loadResource() {/**/}
```

```
1 $ cat main.c
```

```
2 #include "values.h"
```

```
3
```

```
4 struct Resource* loadResource() {/**/}
```

```
5
```

```
6 int main() {
```

```
7     struct Resource* resource = loadResource();
```

```
8     return resource->value;
```

```
9 }
```

```
1 $ nm values.o main.o
```

```
2 values.o:
```

```
3 0000000000000000 W loadResource
```

```
4 main.o:
```

```
5 0000000000000000 T loadResource
```


WEAK SYMBOLS

```
$ cat values.h
#pragma once

struct Resource *loadResource();
```

```
1 $ cat values.c
2 #include "values.h"
3
4 __attribute__((weak)) struct Resource* loadResource() {/**/}
```

```
1 $ cat main.c
2 #include "values.h"
3
4 struct Resource* loadResource() {/**/}
5
6 int main() {
7     struct Resource* resource = loadResource();
8     return resource->value;
9 }
```

```
1 $ nm values.o main.o
2 values.o:
3 0000000000000000 W loadResource
4 main.o:
5 0000000000000000 T loadResource
```


WEAK SYMBOLS

```
$ cat values.h
#pragma once

struct Resource *loadResource();
```

```
1 $ cat values.c
2 #include "values.h"
3
4 __attribute__((weak)) struct Resource* loadResource() {/**/}
```

```
1 $ cat main.c
2 #include "values.h"
3
4 struct Resource* loadResource() {/**/}
5
6 int main() {
7     struct Resource* resource = loadResource();
8     return resource->value;
9 }
```

```
1 $ nm values.o main.o
2 values.o:
3 0000000000000000 W loadResource
4 main.o:
5 0000000000000000 T loadResource
```

INLINE VARIABLES

INLINE VARIABLES

```
1 $ cat values.h
2 #pragma once
3
4 inline int globalValue = 1;
5
6 struct SomeClass {
7     static int getValue() { return localValue; }
8     static inline int localValue = 2;
9 };
```

INLINE VARIABLES

```
1 $ cat values.h
2 #pragma once
3
4 inline int globalValue = 1;
5
6 struct SomeClass {
7     static int getValue() { return localValue; }
8     static inline int localValue = 2;
9 };
```

INLINE VARIABLES

```
1 $ cat values.h
2 #pragma once
3
4 inline int globalValue = 1;
5
6 struct SomeClass {
7     static int getValue() { return localValue; }
8     static inline int localValue = 2;
9 };
```

INLINE VARIABLES

```
1 $ cat values.h
2 #pragma once
3
4 inline int globalValue = 1;
5
6 struct SomeClass {
7     static int getValue() { return localValue; }
8     static inline int localValue = 2;
9 };
```

INLINE VARIABLES

```
1 $ cat values.h
2 #pragma once
3
4 inline int globalValue = 1;
5
6 struct SomeClass {
7     static int getValue() { return localValue; }
8     static inline int localValue = 2;
9 };
```

```
$ cat main.cpp
#include "values.h"

int main() {
    return globalValue + SomeClass::getValue();
}
```


INLINE VARIABLES

```
1 $ cat values.h
2 #pragma once
3
4 inline int globalValue = 1;
5
6 struct SomeClass {
7     static int getValue() { return localValue; }
8     static inline int localValue = 2;
9 };
```

```
$ cat main.cpp
#include "values.h"

int main() {
    return globalValue + SomeClass::getValue();
}
```

```
1 $ nm -C main.o
2 0000000000000000 T main()
3 0000000000000000 u SomeClass::localValue
4 0000000000000000 W SomeClass::getValue()
5 0000000000000000 u globalValue
```


INLINE VARIABLES

```
1 $ cat values.h
2 #pragma once
3
4 inline int globalValue = 1;
5
6 struct SomeClass {
7     static int getValue() { return localValue; }
8     static inline int localValue = 2;
9 };
```

```
$ cat main.cpp
#include "values.h"

int main() {
    return globalValue + SomeClass::getValue();
}
```

```
1 $ nm -C main.o
2 0000000000000000 T main()
3 0000000000000000 u SomeClass::localValue
4 0000000000000000 W SomeClass::getValue()
5 0000000000000000 u globalValue
```

INLINE VARIABLES

```
1 $ cat values.h
2 #pragma once
3
4 inline int globalValue = 1;
5
6 struct SomeClass {
7     static int getValue() { return localValue; }
8     static inline int localValue = 2;
9 };
```

```
$ cat main.cpp
#include "values.h"

int main() {
    return globalValue + SomeClass::getValue();
}
```

```
1 $ nm -C main.o
2 0000000000000000 T main()
3 0000000000000000 u SomeClass::localValue
4 0000000000000000 W SomeClass::getValue()
5 0000000000000000 u globalValue
```

INLINE VARIABLES

```
1 $ cat values.h
2 #pragma once
3
4 inline int globalValue = 1;
5
6 struct SomeClass {
7     static int getValue() { return localValue; }
8     static inline int localValue = 2;
9 };
```

```
$ cat main.cpp
#include "values.h"

int main() {
    return globalValue + SomeClass::getValue();
}
```

```
1 $ nm -C main.o
2 0000000000000000 T main()
3 0000000000000000 u SomeClass::localValue
4 0000000000000000 W SomeClass::getValue()
5 0000000000000000 u globalValue
```

INLINE VARIABLES

```
1 $ cat values.h
2 #pragma once
3
4 inline int globalValue = 1;
5
6 struct SomeClass {
7     static int getValue() { return localValue; }
8     static inline int localValue = 2;
9 };
```

```
$ cat main.cpp
#include "values.h"

int main() {
    return globalValue + SomeClass::getValue();
}
```

```
1 $ nm -C main.o
2 0000000000000000 T main()
3 0000000000000000 u SomeClass::localValue
4 0000000000000000 W SomeClass::getValue()
5 0000000000000000 u globalValue
```

INLINE VARIABLES

```
1 $ cat values.h
2 #pragma once
3
4 inline int globalValue = 1;
5
6 struct SomeClass {
7     static int getValue() { return localValue; }
8     static inline int localValue = 2;
9 };
```

```
$ cat main.cpp
#include "values.h"

int main() {
    return globalValue + SomeClass::getValue();
}
```

```
1 $ nm -C main.o
2 0000000000000000 T main()
3 0000000000000000 u SomeClass::localValue
4 0000000000000000 W SomeClass::getValue()
5 0000000000000000 u globalValue
```

u = unique symbol

INLINE VARIABLES

```
1 $ cat values.h
2 #pragma once
3
4 inline int globalValue = 1;
5
6 struct SomeClass {
7     static int getValue() { return localValue; }
8     static inline int localValue = 2;
9 };
```

```
$ cat main.cpp
#include "values.h"

int main() {
    return globalValue + SomeClass::getValue();
}
```

```
1 $ nm -C main.o
2 0000000000000000 T main()
3 0000000000000000 u SomeClass::localValue
4 0000000000000000 W SomeClass::getValue()
5 0000000000000000 u globalValue
```

u = unique symbol

U = undefined symbol 🥲

LINKING WEAK SYMBOLS

LINKING WEAK SYMBOLS

- I thought the linker just copies sections from object files into the binary

LINKING WEAK SYMBOLS

- I thought the linker just copies sections from object files into the binary
- How can it remove single functions or objects from sections?

LINKING WEAK SYMBOLS

- I thought the linker just copies sections from object files into the binary
- How can it remove single functions or objects from sections?
- It can't!

LINKING WEAK SYMBOLS

- I thought the linker just copies sections from object files into the binary
- How can it remove single functions or objects from sections?
- It can't!

```
1 $ objdump -h main.o | grep .text
2 2 .text 00000048 0000000000000000 0000000000000000 00000050 2**0
3 5 .text._ZN9SomeClass5getValue 0000000f 0000000000000000 0000000000000000 00000000
4 6 .text._Z5getValueIiET_v 0000000b 0000000000000000 0000000000000000 0000000a
```

LINKING WEAK SYMBOLS

- I thought the linker just copies sections from object files into the binary
- How can it remove single functions or objects from sections?
- It can't!

```
1 $ objdump -h main.o | grep .text
2 2 .text          00000048 0000000000000000 0000000000000000 00000050 2**0
3 5 .text._ZN9SomeClass5getValue 0000000f 0000000000000000 0000000000000000 00000000
4 6 .text._Z5getValueIiET_v 0000000b 0000000000000000 0000000000000000 0000000a
```

LINKING WEAK SYMBOLS

- I thought the linker just copies sections from object files into the binary
- How can it remove single functions or objects from sections?
- It can't!

```
1 $ objdump -h main.o | grep .text
2 2 .text 00000048 0000000000000000 0000000000000000 00000050 2**0
3 5 .text._ZN9SomeClass5getValue 0000000f 0000000000000000 0000000000000000 00000000
4 6 .text._Z5getValueIiET_v 0000000b 0000000000000000 0000000000000000 0000000a
```

BINDING

BINDING

- C/C++ External/Internal linkage -> ELF global/local binding

BINDING

- C/C++ External/Internal linkage -> ELF global/local binding
- Weak and unique binding for symbols defined in headers

BINDING

- C/C++ External/Internal linkage -> ELF global/local binding
- Weak and unique binding for symbols defined in headers
- Member functions, function templates, inline variables

BINDING

- C/C++ External/Internal linkage -> ELF global/local binding
- Weak and unique binding for symbols defined in headers
- Member functions, function templates, inline variables
- `__attribute__((weak))` to allow overriding

BINDING

- C/C++ External/Internal linkage -> ELF global/local binding
- Weak and unique binding for symbols defined in headers
- Member functions, function templates, inline variables
- `__attribute__((weak))` to allow overriding
- Linker uses section per weak/unique symbol

DYNAMIC LINKING

DYNAMIC LINKING

- The C++ 23 Standard is 2124 pages

DYNAMIC LINKING

- The C++ 23 Standard is 2124 pages
- How many pages for dynamic linking?

DYNAMIC LINKING

- The C++ 23 Standard is 2124 pages
- How many pages for dynamic linking?
- 0

DYNAMIC LINKING

- The C++ 23 Standard is 2124 pages
- How many pages for dynamic linking?
- 0
- C/C++ do not know about dynamic linking

DYNAMIC LINKING

- The C++ 23 Standard is 2124 pages
- How many pages for dynamic linking?
- 0
- C/C++ do not know about dynamic linking
- All vendor specific

values.c

```
int value = 42;

int getValue() {
    return value;
}
```

other.c

```
#include "values.h"

int other() {
    return getValue();
}
```

values.o

2a	00	00	00
55	48	89	e5
8b	05	00	00
00	00	5d	c3

other.o

0f	d6	23	04
11	f8	d1	c3
87	03	12	7a
3b	48	28	23
fe	e9	63	c6

library.a

83	75	03	4f
8f	20	a7	37
83	74	9c	39
27	37	ff	a8
34	9c	61	23
87	fa	ca	72
94	17	53	93
9b	be	38	48

the_lib.so

2a	00	00	00
0f	d6	23	04
83	75	03	4f
8f	20	a7	37
55	48	89	e5
8b	05	00	00
00	00	5d	c3
11	f8	d1	c3
87	03	12	7a
3b	48	28	23
fe	e9	63	c6
83	74	9c	39
27	37	ff	a8
34	9c	61	23
87	fa	ca	72
94	17	53	93
9b	be	38	48

DYNAMIC SHARED OBJECT

```
//library.h  
#pragma once  
  
int internal_function();  
int api_function();
```

DYNAMIC SHARED OBJECT

```
//library.h  
#pragma once
```

```
int internal_function();  
int api_function();
```

```
//library.c  
#include "library.h"
```

```
int internal_function() { return 1; }  
int api_function() { return internal_function() * 2; }
```

DYNAMIC SHARED OBJECT

```
//library.h  
#pragma once
```

```
int internal_function();  
int api_function();
```

```
//library.c  
#include "library.h"
```

```
int internal_function() { return 1; }  
int api_function() { return internal_function() * 2; }
```

```
$ nm library.o  
0000000000000000 T api_function  
000000000000000b T internal_function
```

DYNAMIC SHARED OBJECT

```
//library.h  
#pragma once
```

```
int internal_function();  
int api_function();
```

```
//library.c  
#include "library.h"
```

```
int internal_function() { return 1; }  
int api_function() { return internal_function() * 2; }
```

```
$ nm library.o  
0000000000000000 T api_function  
000000000000000b T internal_function
```

```
$ nm libdynlib.so  
(...)  
000000000000005ba T api_function  
000000000000005c5 T internal_function
```


VISIBILITY VS. BINDING

- Exposed symbols in DSO controlled by visibility

VISIBILITY VS. BINDING

- Exposed symbols in DSO controlled by visibility
- Visibility $\neq$ binding

VISIBILITY VS. BINDING

- Exposed symbols in DSO controlled by visibility
- Visibility $\neq$ binding
- Binding: Can the symbol be used outside the TU

VISIBILITY VS. BINDING

- Exposed symbols in DSO controlled by visibility
- Visibility $\neq$ binding
- Binding: Can the symbol be used outside the TU
- Binding matters when creating the library

VISIBILITY VS. BINDING

- Exposed symbols in DSO controlled by visibility
- Visibility $\neq$ binding
- Binding: Can the symbol be used outside the TU
- Binding matters when creating the library
- Visibility: Can the symbol eventually be used outside the DSO

VISIBILITY VS. BINDING

- Exposed symbols in DSO controlled by visibility
- Visibility $\neq$ binding
- Binding: Can the symbol be used outside the TU
- Binding matters when creating the library
- Visibility: Can the symbol eventually be used outside the DSO
- Visibility matters when linking to the library

VISIBILITY VS. BINDING

- C/C++ do not know dynamic linking or visibility!

VISIBILITY VS. BINDING

- C/C++ do not know dynamic linking or visibility!
- Purely vendor specific

VISIBILITY VS. BINDING

- C/C++ do not know dynamic linking or visibility!
- Purely vendor specific
- On Linux: Default vs hidden visibility

VISIBILITY VS. BINDING

- C/C++ do not know dynamic linking or visibility!
- Purely vendor specific
- On Linux: Default vs hidden visibility
 - `__attribute__((visibility(default|hidden)))`

VISIBILITY VS. BINDING

- C/C++ do not know dynamic linking or visibility!
- Purely vendor specific
- On Linux: Default vs hidden visibility
 - `__attribute__((visibility(default|hidden)))`
 - `-fvisibility=hidden`

VISIBILITY VS. BINDING

- C/C++ do not know dynamic linking or visibility!
- Purely vendor specific
- On Linux: Default vs hidden visibility
 - `__attribute__((visibility(default|hidden)))`
 - `-fvisibility=hidden`
- Default is default visibility

VISIBILITY VS. BINDING

- C/C++ do not know dynamic linking or visibility!
- Purely vendor specific
- On Linux: Default vs hidden visibility
 - `__attribute__((visibility(default|hidden)))`
 - `-fvisibility=hidden`
- Default is default visibility
- On Windows: `__declspec(dllexport|dllimport)`

CONTROLLING VISIBILITY

```
#CMakeLists.txt  
add_library(dynlib SHARED library.c)  
target_compile_options(dynlib PRIVATE -fvisibility=hidden)
```

CONTROLLING VISIBILITY

```
#CMakeLists.txt
add_library(dynlib SHARED library.c)
target_compile_options(dynlib PRIVATE -fvisibility=hidden)
```

```
//library.h
#pragma once
int internal_function();
__attribute__((visibility("default"))) int api_function();
```

CONTROLLING VISIBILITY

```
#CMakeLists.txt
add_library(dynlib SHARED library.c)
target_compile_options(dynlib PRIVATE -fvisibility=hidden)
```

```
#define DSO_EXPORT __attribute__((visibility("default")))
```

```
//library.h
#pragma once
int internal_function();
__attribute__((visibility("default"))) int api_function();
```


CONTROLLING VISIBILITY

```
#CMakeLists.txt
add_library(dynlib SHARED library.c)
target_compile_options(dynlib PRIVATE -fvisibility=hidden)
```

```
#define DSO_EXPORT __attribute__((visibility("default")))
```

```
//library.h
#pragma once
int internal_function();
DSO_EXPORT int api_function();
```


CROSS PLATFORM

CROSS PLATFORM

Windows: `dllexport/dllimport`

CROSS PLATFORM

Windows: `dllexport/dllimport`

```
1 #if defined _WIN32
2     #ifdef BUILDING_DLL
3         #define DSO_EXPORT __declspec(dllexport)
4     #else
5         #define DSO_EXPORT __declspec(dllimport)
6     #endif
7 #else
8     #define DSO_EXPORT __attribute__((visibility ("default")))
9 #endif
```

CROSS PLATFORM

Windows: `dllexport/dllimport`

```
1 #if defined _WIN32
2     #ifdef BUILDING_DLL
3         #define DSO_EXPORT __declspec(dllexport)
4     #else
5         #define DSO_EXPORT __declspec(dllimport)
6     #endif
7 #else
8     #define DSO_EXPORT __attribute__((visibility ("default")))
9 #endif
```

CROSS PLATFORM

Windows: `dllexport/dllimport`

```
1 #if defined _WIN32
2     #ifdef BUILDING_DLL
3         #define DSO_EXPORT __declspec(dllexport)
4     #else
5         #define DSO_EXPORT __declspec(dllimport)
6     #endif
7 #else
8     #define DSO_EXPORT __attribute__((visibility ("default")))
9 #endif
```

VISIBILITY VS BINDING

```
$ nm library.o
0000000000000000b T api_function
00000000000000000 T internal_function
```

VISIBILITY VS BINDING

```
$ nm library.o
0000000000000000b T api_function
0000000000000000 T internal_function
```

```
$ readelf -s library.o
```

Symbol table '.symtab' contains 15 entries:

Num:	Value	Size	Type	Bind	Vis	Ndx	Name
(...)							
13:	0000000000000000	11	FUNC	GLOBAL	HIDDEN	1	internal_function
14:	0000000000000000b	18	FUNC	GLOBAL	DEFAULT	1	api_function

VISIBILITY VS BINDING

```
$ nm library.o
0000000000000000b T api_function
00000000000000000 T internal_function
```

```
$ readelf -s library.o
```

Symbol table '.symtab' contains 15 entries:

Num:	Value	Size	Type	Bind	Vis	Ndx	Name
(...)							
13:	00000000000000000	11	FUNC	GLOBAL	HIDDEN	1	internal_function
14:	0000000000000000b	18	FUNC	GLOBAL	DEFAULT	1	api_function

```
$ readelf -s libdynlib.so
```

Symbol table '.dynsym' contains 11 entries:

Num:	Value	Size	Type	Bind	Vis	Ndx	Name
(...)							
5:	00000000000000595	18	FUNC	GLOBAL	DEFAULT	9	api_function

Symbol table '.symtab' contains 54 entries:

Num:	Value	Size	Type	Bind	Vis	Ndx	Name
(...)							
41:	0000000000000058a	11	FUNC	LOCAL	DEFAULT	9	internal_function
45:	00000000000000595	18	FUNC	GLOBAL	DEFAULT	9	api_function

WHY TWO SYMBOL TABLES?

- Why not put binding and visibility in one table?

WHY TWO SYMBOL TABLES?

- Why not put binding and visibility in one table?
- Dynamic symbol table is needed at run-time

WHY TWO SYMBOL TABLES?

- Why not put binding and visibility in one table?
- Dynamic symbol table is needed at run-time
- Consumer must adjust the addresses based on where DSO was loaded

WHY TWO SYMBOL TABLES?

- Why not put binding and visibility in one table?
- Dynamic symbol table is needed at run-time
- Consumer must adjust the addresses based on where DSO was loaded
- Nice if small to load and look up in

WHY TWO SYMBOL TABLES?

- Why not put binding and visibility in one table?
- Dynamic symbol table is needed at run-time
- Consumer must adjust the addresses based on where DSO was loaded
- Nice if small to load and look up in
- Static symbol table can be removed with `strip` after linking!

WHY ~~TWO~~ THREE SYMBOL TABLES?

- Why not put binding and visibility in one table?
- Dynamic symbol table is needed at run-time
- Consumer must adjust the addresses based on where DSO was loaded
- Nice if small to load and look up in
- Static symbol table can be removed with `strip` after linking!

WHY ~~TWO~~ THREE SYMBOL TABLES?

- Why not put binding and visibility in one table?
- Dynamic symbol table is needed at run-time
- Consumer must adjust the addresses based on where DSO was loaded
- Nice if small to load and look up in
- Static symbol table can be removed with `strip` after linking!
- Debug info can be huge, is optional, and can be separately stripped

WHO GETS A SYMBOL


```
int function(  
    int param)  
{  
    int value = 1;  
    return value + param;  
}
```



```
int function(          // T
    int param)        //
{
    int value = 1;     //
    return value + param;
}
```



```
int function(  
    int param)  
{  
    static int value = 1;  
    return value + param;  
}
```

```
int function(          //T
    int param)
{
    static int value = 1; //d
    return value + param;
}
```

```
struct Struct {  
    int member;  
  
    int function() { return 1; }  
};
```

```
struct Struct {                                //  
    int member;                                //  
  
    int function() { return 1; }              //W?  
};
```

```
struct Struct {  
    static int member;  
  
    static int function() { return 1; }  
};
```

```
struct Struct {  
    static int member;           // U?  
  
    static int function() { return 1; } // W?  
};
```

WHO GETS A SYMBOL?

WHO GETS A SYMBOL?

- Only "things there are one of" get symbols

WHO GETS A SYMBOL?

- Only "things there are one of" get symbols
- If a symbol has internal linkage and is inlined , it's omitted

WHO GETS A SYMBOL?

- Only "things there are one of" get symbols
- If a symbol has internal linkage and is inlined , it's omitted
- If a weak/unique symbol is unused, it's omitted

SUMMARY

SUMMARY

- Separate compilation model, relocations

SUMMARY

- Separate compilation model, relocations
- Symbols for linker to find things

SUMMARY

- Separate compilation model, relocations
- Symbols for linker to find things
- External/Internal linkage -> global/local binding

SUMMARY

- Separate compilation model, relocations
- Symbols for linker to find things
- External/Internal linkage -> global/local binding
- Weak and unique binding for symbols defined in headers

SUMMARY

- Separate compilation model, relocations
- Symbols for linker to find things
- External/Internal linkage -> global/local binding
- Weak and unique binding for symbols defined in headers
- Visibility for shared objects

SUMMARY

- Separate compilation model, relocations
- Symbols for linker to find things
- External/Internal linkage -> global/local binding
- Weak and unique binding for symbols defined in headers
- Visibility for shared objects
- `-fvisibility / __attribute__((visibility))`

SUMMARY

- Separate compilation model, relocations
- Symbols for linker to find things
- External/Internal linkage -> global/local binding
- Weak and unique binding for symbols defined in headers
- Visibility for shared objects
- `-fvisibility / __attribute__((visibility))`
- Sections: `.data`, `.text`, `.symtab`, `.dynsym`, `.rela*`

SUMMARY

- Separate compilation model, relocations
- Symbols for linker to find things
- External/Internal linkage -> global/local binding
- Weak and unique binding for symbols defined in headers
- Visibility for shared objects
- `-fvisibility / __attribute__((visibility))`
- Sections: `.data`, `.text`, `.symtab`, `.dynsym`, `.rela*`
- `nm`, `objdump`, `readelf`

SUMMARY

- Separate compilation model, relocations
- Symbols for linker to find things
- External/Internal linkage -> global/local binding
- Weak and unique binding for symbols defined in headers
- Visibility for shared objects
- `-fvisibility / __attribute__((visibility))`
- Sections: `.data`, `.text`, `.symtab`, `.dynsym`, `.rela*`
- `nm`, `objdump`, `readelf`
- `strip` can remove symbols

SUMMARY

- Separate compilation model, relocations
- Symbols for linker to find things
- External/Internal linkage -> global/local binding
- Weak and unique binding for symbols defined in headers
- Visibility for shared objects
- `-fvisibility / __attribute__((visibility))`
- Sections: `.data`, `.text`, `.symtab`, `.dynsym`, `.rela*`
- `nm`, `objdump`, `readelf`
- `strip` can remove symbols
- "Singular" things get symbols

HOW SYMBOLS WORK AND WHY WE NEED THEM

ANDERS SCHAU KNATTEN

C++ ON SEA 2024



Just Enough Assembly for
Compiler Explorer

What Happens After
the Compiler



